



COMPLEX COMPUTING PROBLEM (CCP)

Field	Detail
Student Name	Hassan Bahar
Registration No.	F2023266769
Total Marks	10 (CLO-3, CLO-4, CLO-5)
Submission Mode	Code + Technical Report
Submission Date	08/Jul/2026

**University of Management
And Technology**

Table of Contents

2. Pipeline Architecture.....	4
3. Task 1 — Edge Detection & Hough Transform (CLO-3).....	5
3.1 Objective.....	5
3.2 Methodology.....	5
<i>Step 1 — Gaussian Smoothing</i>	5
<i>Step 2 — Canny Edge Detection: Threshold Tuning</i>	5
<i>Step 3 — Probabilistic Hough Line Transform</i>	5
3.3 Results.....	6
3.4 Discussion	7
4.Task 2 — SIFT Feature Extraction & Matching (CLO-3)	8
4.1 Objective.....	8
4.2 Methodology.....	8
<i>Step 1 — SIFT Feature Extraction</i>	8
<i>Step 2 — Descriptor Matching and Lowe's Ratio Test</i>	8
<i>Step 3 — Homography Estimation & RANSAC Outlier Rejection</i>	8
4.3 Results.....	9
.....	9
Pano Of 3 images	11
.....	11
4.4 Discussion	11
5. Task 3 — Image Stitching / Panorama Construction (CLO-4).....	12
5.1 Objective.....	12
5.2 Methodology.....	12
<i>Step 1 — Two-Stage Stitching Strategy</i>	12
<i>Step 2 — Feature Matching (Stage 2: Intermediate Panorama + Right Frame)</i>	12
<i>Step 3 — Warping, Cropping and Seam Correction</i>	12
5.3 Results.....	13
5.4 Discussion	14
6. Task 4 — CNN-Based Scene Classification (CLO-4)	15
6.1 Objective.....	15
6.2 Dataset	15
6.3 Preprocessing & Augmentation	15
6.4 Model Architecture & Training Configuration	17
6.5 Results.....	17

Per-Class Precision, Recall and F1-score 17

6.6 Grad-CAM Interpretability 19

6.7 Discussion 20

6.8 Bonus: Interactive Deployment (Gradio UI)..... 20

5 — Critical Evaluation of the Pipeline (CLO-5) 21

7.1 Quantitative Summary Across All Tasks 21

7.2 Failure Analysis (≥ 2 modes per task)..... 21

Task 1 — Edge Detection & Hough Transform..... 21

Task 2 — SIFT Feature Matching 21

Task 3 — Panorama Construction 21

Task 4 — CNN Scene Classification 22

7.3 Classical vs. Deep-Learning Trade-offs (Tasks 1–3 vs. Task 4) 22

7.4 Limitations & Future Work 22

8. Tools Used & Academic Integrity Declaration **Error! Bookmark not defined.**

1. Executive Summary

This report documents the design, implementation, and critical evaluation of an Autonomous Visual Intelligence System developed for the CS-456 Computer Vision Complex Computing Problem (CCP). The system was built as a four-stage pipeline that combines classical image-processing techniques with a deep-learning classifier to emulate the perception stack of an outdoor navigation robot.

The pipeline first detects structural edges and road boundaries using Gaussian smoothing, Canny edge detection, and the Probabilistic Hough Transform (Task 1). It then establishes visual correspondences between images of the same scene captured from different viewpoints using SIFT feature extraction and Brute-Force matching with Lowe's ratio test (Task 2). These correspondences are reused, together with RANSAC-based homography estimation, to stitch multiple overlapping frames into a single panoramic map through manual perspective warping, cropping, and seam-blending — implemented entirely without OpenCV's built-in `cv2.Stitcher` API (Task 3). Finally, a MobileNetV2 network fine-tuned via transfer learning classifies scenes into six outdoor categories (buildings, forest, glacier, mountain, sea, street), reaching a held-out test accuracy of 91.67%, and its decisions are interpreted using Grad-CAM visualizations (Task 4). As an additional extension beyond the CCP's minimum scope, the trained model was also deployed behind an interactive Gradio web interface that lets a user upload an image and receive a live scene prediction with per-class confidence scores.

Section 5 closes the report with a critical, quantitative evaluation of the complete pipeline: it tabulates the key performance metrics for every stage, analyses at least two concrete failure modes per task, compares the classical (Tasks 1–3) and deep-learning (Task 4) approaches on speed, accuracy, interpretability and data requirements, and discusses the limitations and future improvements of the system.

2. Pipeline Architecture

Figure 1 below summarises the end-to-end architecture of the Autonomous Visual Intelligence System. Raw camera frames feed two parallel classical branches — an edge/line-detection branch (Task 1) and a feature-correspondence branch (Task 2) whose matched keypoints are reused for panorama construction (Task 3) — as well as a deep-learning branch that performs scene classification directly on the input frames (Task 4). The outputs of all branches (detected road lines, a stitched panoramic map, and a scene-category label with a Grad-CAM interpretability map) are combined into the final scene-understanding output, which is then critically evaluated in Task 5.

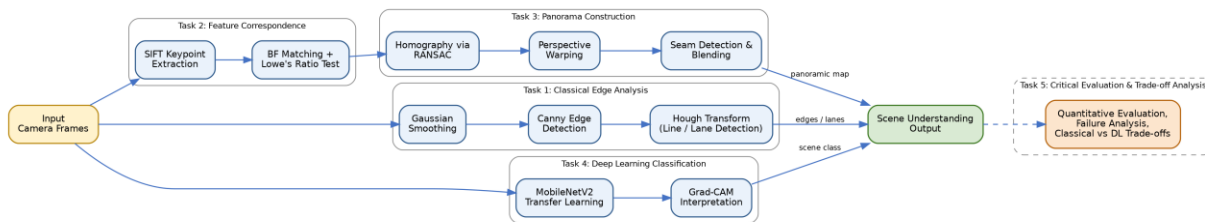


Figure 1: Block diagram of the four-stage Autonomous Visual Intelligence pipeline and its evaluation loop.

3. Task 1 — Edge Detection & Hough Transform (CLO-3)

3.1 Objective

The objective of this task was to detect structural edges and road boundaries in outdoor images using a classical computer-vision pipeline: Gaussian smoothing, Canny edge detection with systematic threshold tuning, and the Probabilistic Hough Line Transform for lane/structure line detection.

3.2 Methodology

Step 1 — Gaussian Smoothing

Each input image was converted to grayscale and smoothed with a Gaussian filter to suppress noise before edge detection.

Parameter	Value
Kernel Size	5×5
Sigma (σ)	1.4

Step 2 — Canny Edge Detection: Threshold Tuning

Three source images were processed end-to-end: a high-resolution road scene (road.jpg, 5472×3648), a second road image (img2, 900×600), and a third scene (image3.jpg, 1174×600). For every image, three Canny threshold configurations were tested on the same Gaussian-blurred frame to study the accuracy/noise trade-off.

Experiment	Low Threshold	High Threshold	Qualitative Observation
Exp-1	50	150	Many noisy / spurious edges retained
Exp-2	100	200	Best balance — clean, continuous road-boundary edges
Exp-3	150	250	Cleaner but weak/faint lane markings are lost

Based on this comparison, the (100, 200) configuration was selected as the operating point for all subsequent Hough-line experiments on the two lower-resolution images.

Step 3 — Probabilistic Hough Line Transform

The Probabilistic Hough Transform was applied to the Canny(100,200) edge maps to recover straight line segments corresponding to lane markings and structural boundaries. Three separate configurations were tested across the two Hough-enabled images, including one run restricted to a triangular/trapezoidal region-of-interest (ROI) mask covering only the expected road corridor.

Image / Run	rho	theta	threshold	minLineLength	maxLineGap	Lines Detected
Image 2 — full frame	1	$\pi/180$	40	80	50	253
Image 2 — ROI-masked	1	$\pi/180$	20	40	100	54
Image 3 — full frame	1	$\pi/180$	50	100	50	28

Applying a road-shaped ROI mask before running the Hough transform is the key parameter-tuning insight of this task: on the full, unmasked edge map, 253 lines were detected, a large share of which come from background structures (buildings, foliage, sky-line clutter) rather than the road itself. Restricting the search to the trapezoidal driving corridor — while simultaneously relaxing the threshold and maxLineGap to compensate for the smaller search area — reduced the false-positive line count to 54, almost all of which correspond to genuine lane boundaries.

3.3 Results

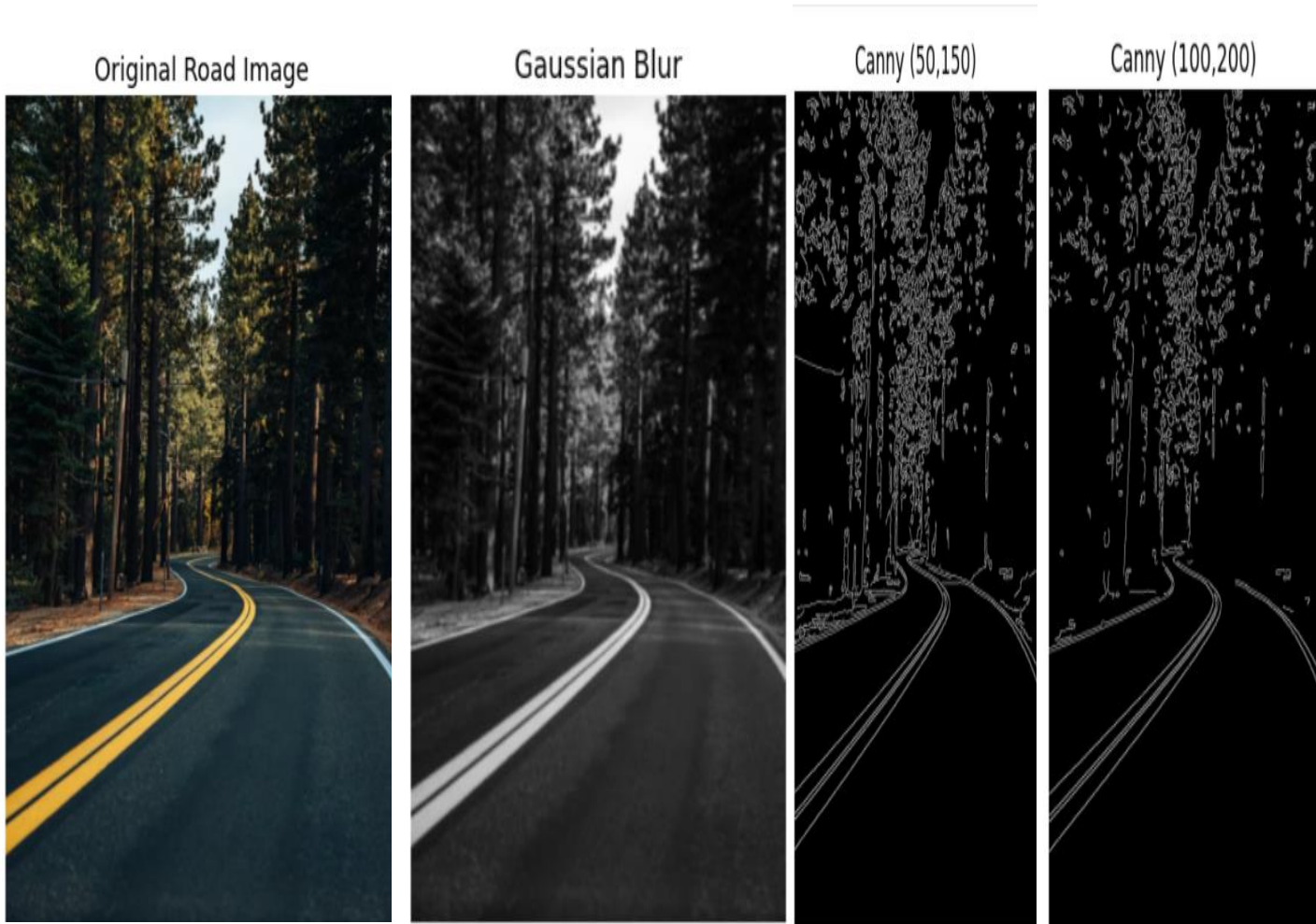


Figure 2: Original image vs. Canny edges vs. Hough-line overlay for the road scene.

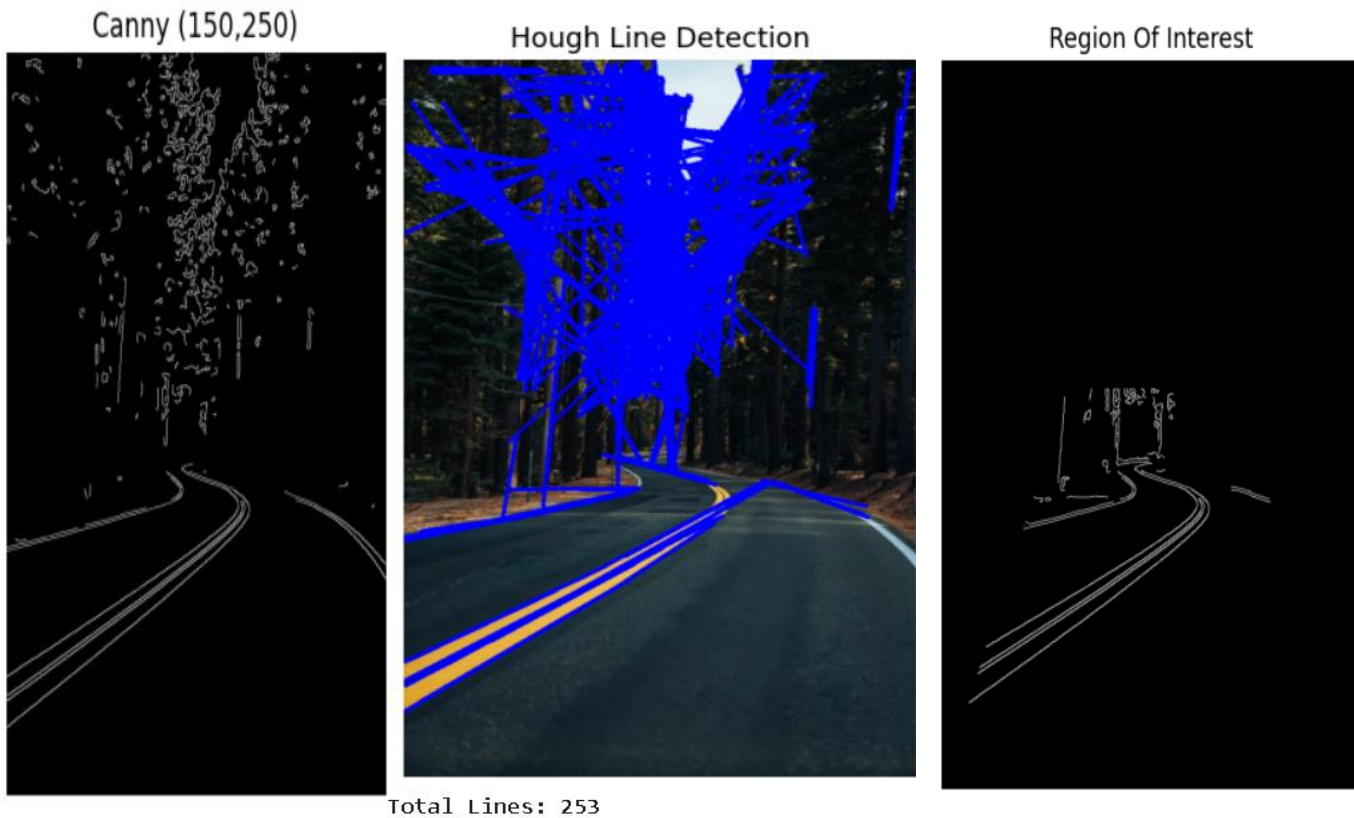


Figure 3: Effect of Canny threshold tuning on edge quality.

Figure 4: ROI-masked Hough transform result, showing reduced false-positive lines.

3.4 Discussion

- Low Canny thresholds (50/150) retain noisy, non-structural edges that later translate into spurious Hough lines.
- High thresholds (150/250) suppress genuine but low-contrast lane markings, causing missed detections.
- The moderate (100,200) configuration gave the best qualitative trade-off across all three test images.
- Hough transform output is extremely sensitive to the quality of the input edge map — the same threshold/gap settings behave very differently on full-frame vs. ROI-masked inputs.
- Region-of-interest masking is an effective, low-cost technique for suppressing background clutter without any learning-based components.

4.Task 2 — SIFT Feature Extraction & Matching (CLO-3)

4.1 Objective

The objective of this task was to establish reliable feature correspondences between two images of the same scene captured from different viewpoints, using SIFT descriptors, Brute-Force matching, and Lowe's ratio test.

4.2 Methodology

Step 1 — SIFT Feature Extraction

SIFT was used to detect scale- and rotation-invariant keypoints in both images of the test pair.

Image	Number of Keypoints
Image 1	81,956
Image 2	18,171

Step 2 — Descriptor Matching and Lowe's Ratio Test

Descriptors were matched with a Brute-Force matcher using k-nearest-neighbours ($k=2$), and Lowe's ratio test (threshold = 0.75) was applied to reject ambiguous matches.

Metric	Value
Total Candidate Pairs (knnMatch)	81,956
Good Matches (after Lowe ratio 0.75)	4,130
Match Quality Ratio (good / total keypoints)	5.04%

Step 3 — Homography Estimation & RANSAC Outlier Rejection

A homography was estimated from the good matches using RANSAC, which simultaneously rejects matches that are not geometrically consistent with a single planar transform.

Metric	Value
Inliers	2,799
Outliers	1,331
RANSAC Inlier Ratio	67.78%

Estimated homography matrix (Image 1 $\rightarrow$ Image 2):

$$H = \begin{bmatrix} 0.6765 & -0.0078 & -902.58 & 0.0608 & 0.6079 & -61.83 & 0.0000968 & -0.0000114 & 1.0000 \end{bmatrix}$$

4.3 Results

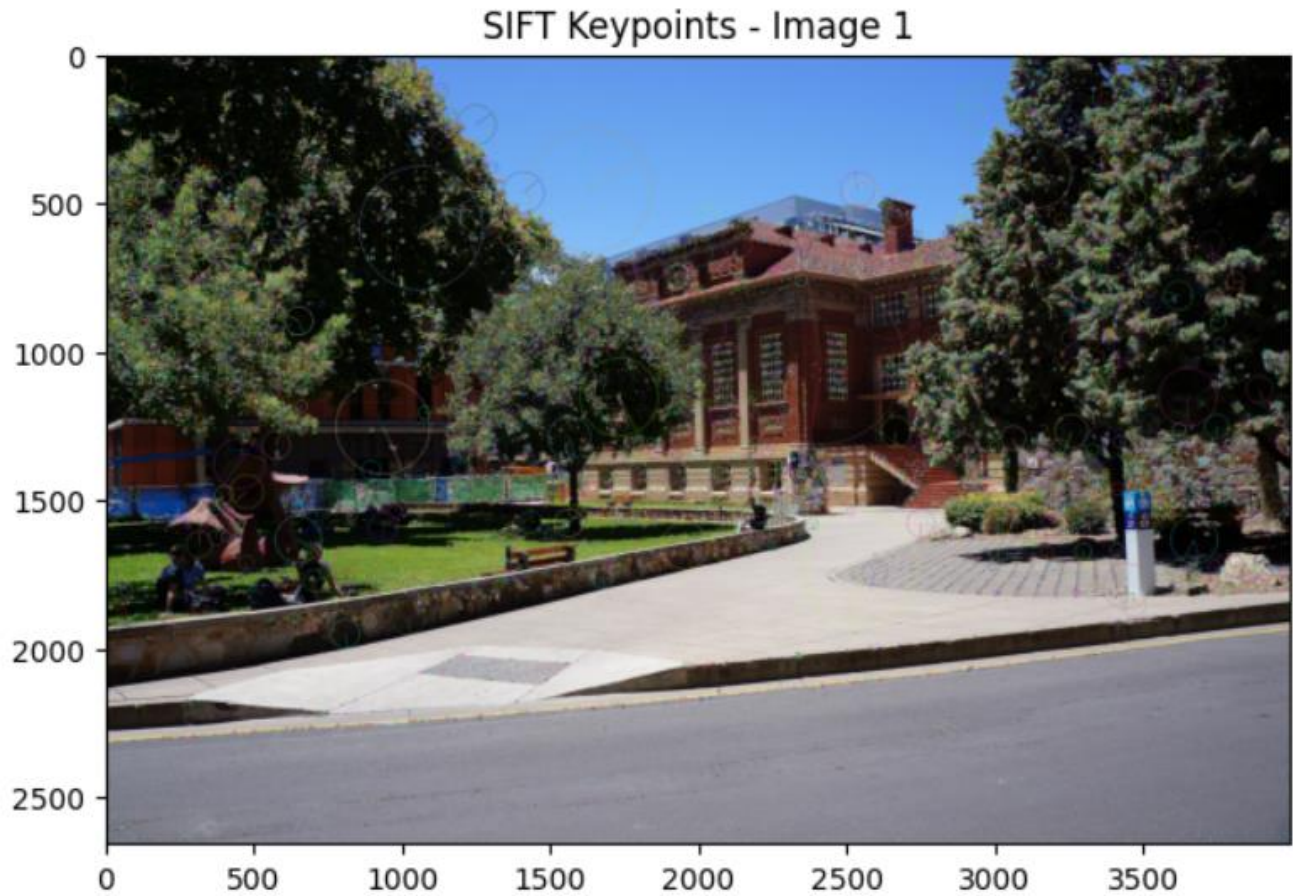


Figure 5: SIFT keypoint visualization — Image 1.

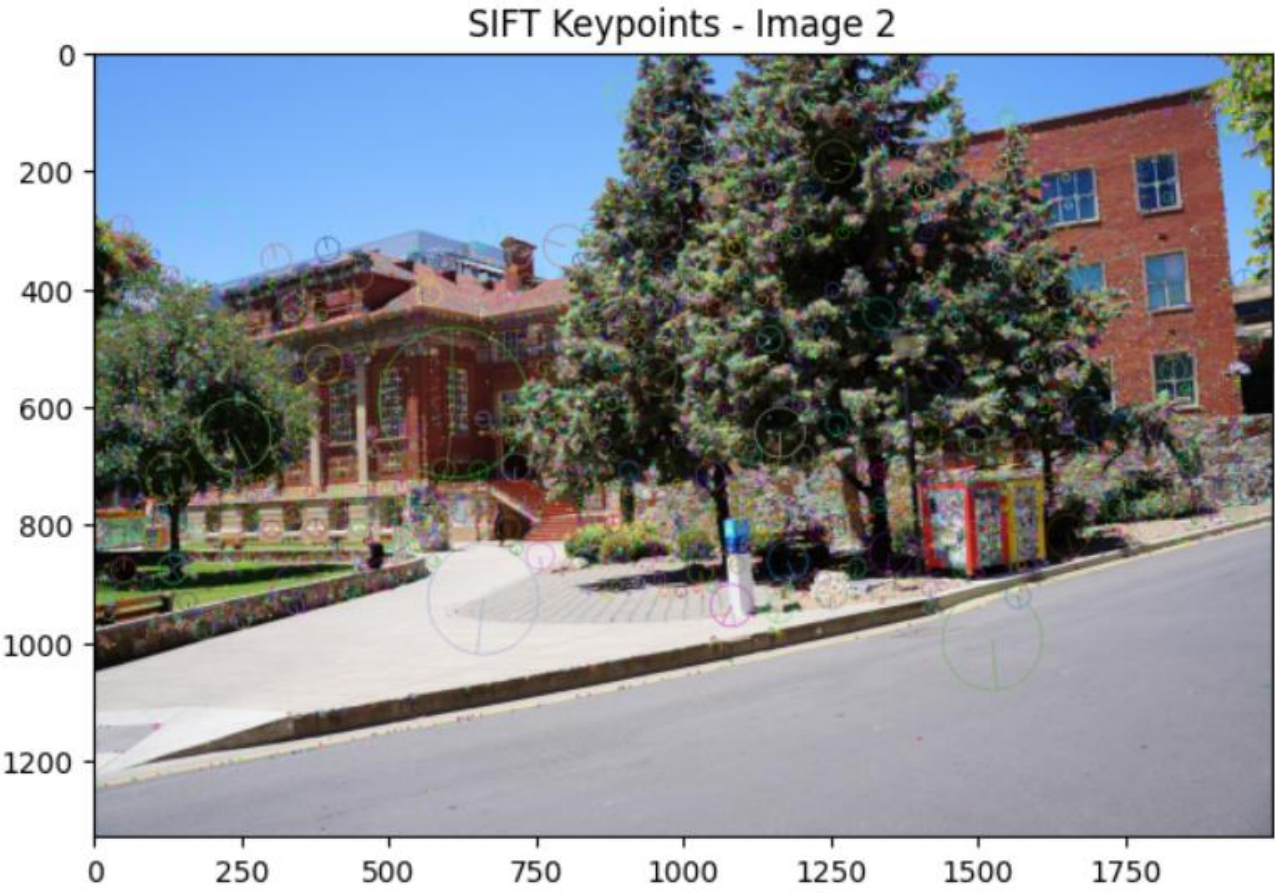


Figure 6: SIFT keypoint visualization — Image 2.

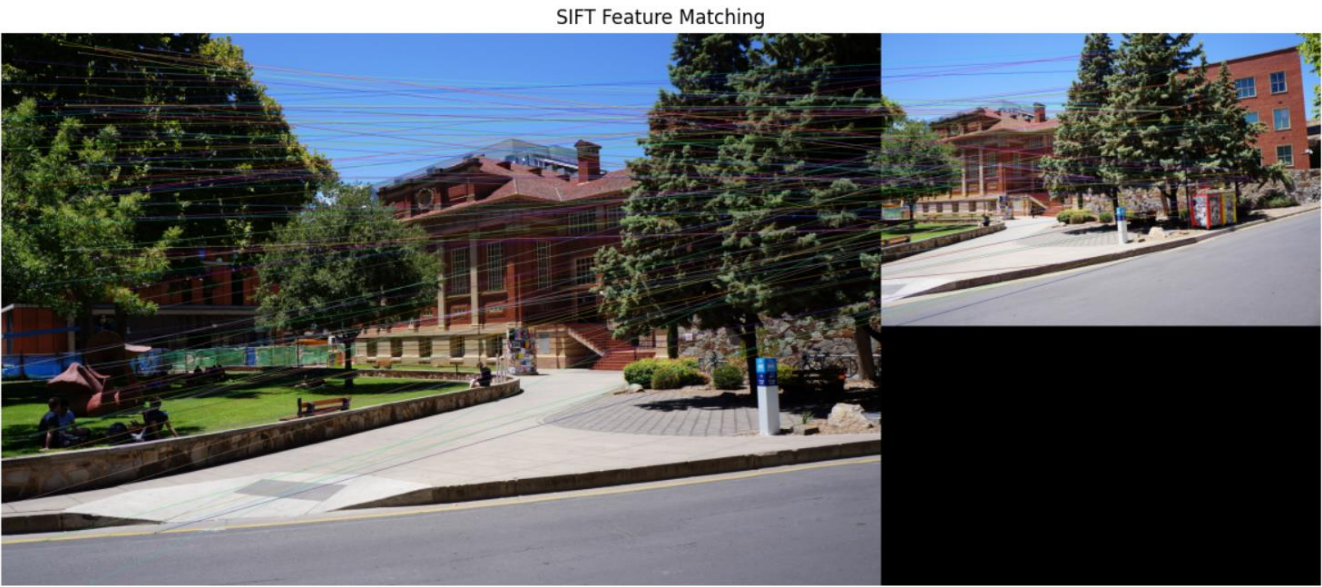


Figure 7: Top-50 SIFT feature matches after Lowe's ratio test.

Final Panorama



Pano Of 3 images



4.4 Discussion

- SIFT remained robust to viewpoint and scale differences even though the two images produced very unequal raw keypoint counts (81,956 vs. 18,171), reflecting a difference in scene texture/resolution.
- Lowe's ratio test discarded the vast majority of ambiguous candidate pairs, keeping only 4,130 of 81,956 (5.04%) as "good" matches — a tight but reliable filter.
- Even after ratio-test filtering, RANSAC still rejected 1,331 of 4,130 good matches (32.2%) as geometric outliers, showing that the ratio test alone does not guarantee geometric consistency, particularly in repetitive or textured regions.
- A tighter ratio threshold generally increases precision (fewer false matches) at the cost of recall (fewer total matches available for the next stage).

5. Task 3 — Image Stitching / Panorama Construction (CLO-4)

5.1 Objective

The objective of this task was to construct a seamless panoramic image from three overlapping frames (left, center, right) using SIFT features, RANSAC-verified homography estimation, perspective warping, and manual seam blending — implemented without any pre-built stitching API such as cv2.Stitcher, per the CCP's constraints.

5.2 Methodology

Step 1 — Two-Stage Stitching Strategy

The panorama was built in two stages: (1) the left and center frames were matched and warped into an intermediate panorama, which was then cropped to remove empty perspective-warp regions (resulting size 2060×1010); (2) this intermediate panorama was then matched and stitched against the right frame to produce the final three-image panorama.

Step 2 — Feature Matching (Stage 2: Intermediate Panorama + Right Frame)

Image	Keypoints
Intermediate Panorama (Left+Center)	12,453
Right Frame	11,527

Metric	Value
Total Good Matches (Lowe ratio 0.75)	318
RANSAC Inliers	265
RANSAC Outliers	53
RANSAC Inlier Ratio	83.33%

Estimated homography matrix (Intermediate Panorama → Right Frame):

$$H = \begin{bmatrix} 2.5537 & -0.2199 & -2115.26 & 0.6411 & 2.1165 & -750.79 & 0.001021 & -0.0000391 & 1.0000 \end{bmatrix}$$

Step 3 — Warping, Cropping and Seam Correction

The right frame was warped into the intermediate panorama's coordinate frame using the estimated homography, and the combined canvas was cropped to [750:1770, 260:4550] to remove black perspective-warp borders, giving a final panorama of approximately 3709 × 1020 pixels.

The seam position was located automatically by searching the middle 50% of the panorama width for the column with the largest mean horizontal intensity gradient, which was found at x = 2172. A brightness/colour discrepancy was then measured across a 150-pixel window straddling the seam:

Channel	Right – Left Mean Difference
R	−39.3
G	−19.1
B	+2.8

This confirms a visible exposure mismatch between the source photographs (the right side is darker in red and green but marginally brighter in blue). The seam region was corrected using local brightness matching and Gaussian-smoothed blending over the transition zone to reduce the visible discontinuity.

5.3 Results

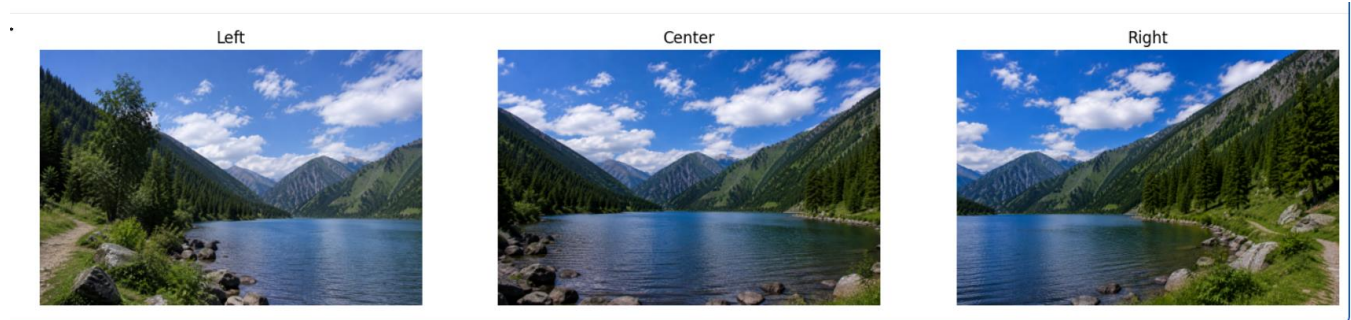


Figure 8: Input frames for the panorama pipeline.



Figure 9: Initial stitched panorama prior to crop.



Figure 10: Final seam-corrected panoramic output.

Metric	Value
Total Matches (final stitch stage)	318
Inliers	265
Outliers	53
Inlier Ratio	83.33%
Final Seam Position	$x = 2172$ px
Final Panorama Size	$\approx 3709 \times 1020$ px

5.4 Discussion

- RANSAC achieved a high 83.33% inlier ratio at the final stitching stage, indicating that the two frames shared a strong, geometrically consistent overlap region.
- Perspective warping introduces unavoidable black border regions whose extent depends on the homography's distortion — these had to be removed via manually tuned crop coordinates rather than an automatic content-aware crop.
- A measurable exposure difference across the seam ($\Delta R \approx -39$, $\Delta G \approx -19$) shows that the classical pipeline, unlike a learned model, has no inherent tolerance to uncontrolled lighting changes between shots and needs an explicit blending/correction step.
- Implementing the pipeline manually (SIFT + BFMatcher + RANSAC + warpPerspective + custom crop/blend), without cv2.Stitcher, satisfies the CCP's technical constraint and makes every failure mode (borders, seams) directly attributable to a specific pipeline stage.

6. Task 4 — CNN-Based Scene Classification (CLO-4)

6.1 Objective

The objective of this task was to design and train a deep-learning-based scene classifier capable of recognising outdoor environments relevant to autonomous navigation, using transfer learning on a pre-trained MobileNetV2 backbone, and to interpret its decisions using Grad-CAM.

6.2 Dataset

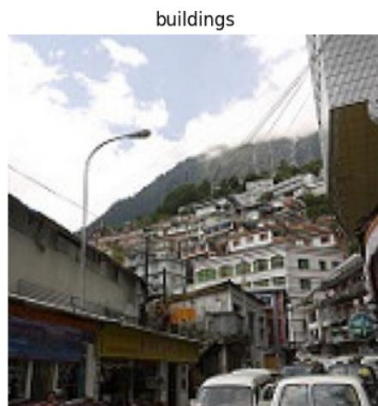
The Intel Image Classification dataset was used, containing six outdoor scene categories: Buildings, Forest, Glacier, Mountain, Sea, and Street.

Split	Number of Images
Training	11,266
Validation	2,815
Testing	3,000

6.3 Preprocessing & Augmentation

- All images resized to 224×224 pixels and passed through the MobileNetV2 preprocessing function.
- Data augmentation applied on the training split: horizontal flip, rotation ($\pm 20^\circ$), zoom (0.2), and brightness jitter (0.8–1.2).

Original Images (Before Preprocessing)





Training Images After Preprocessing & Data Augmentation

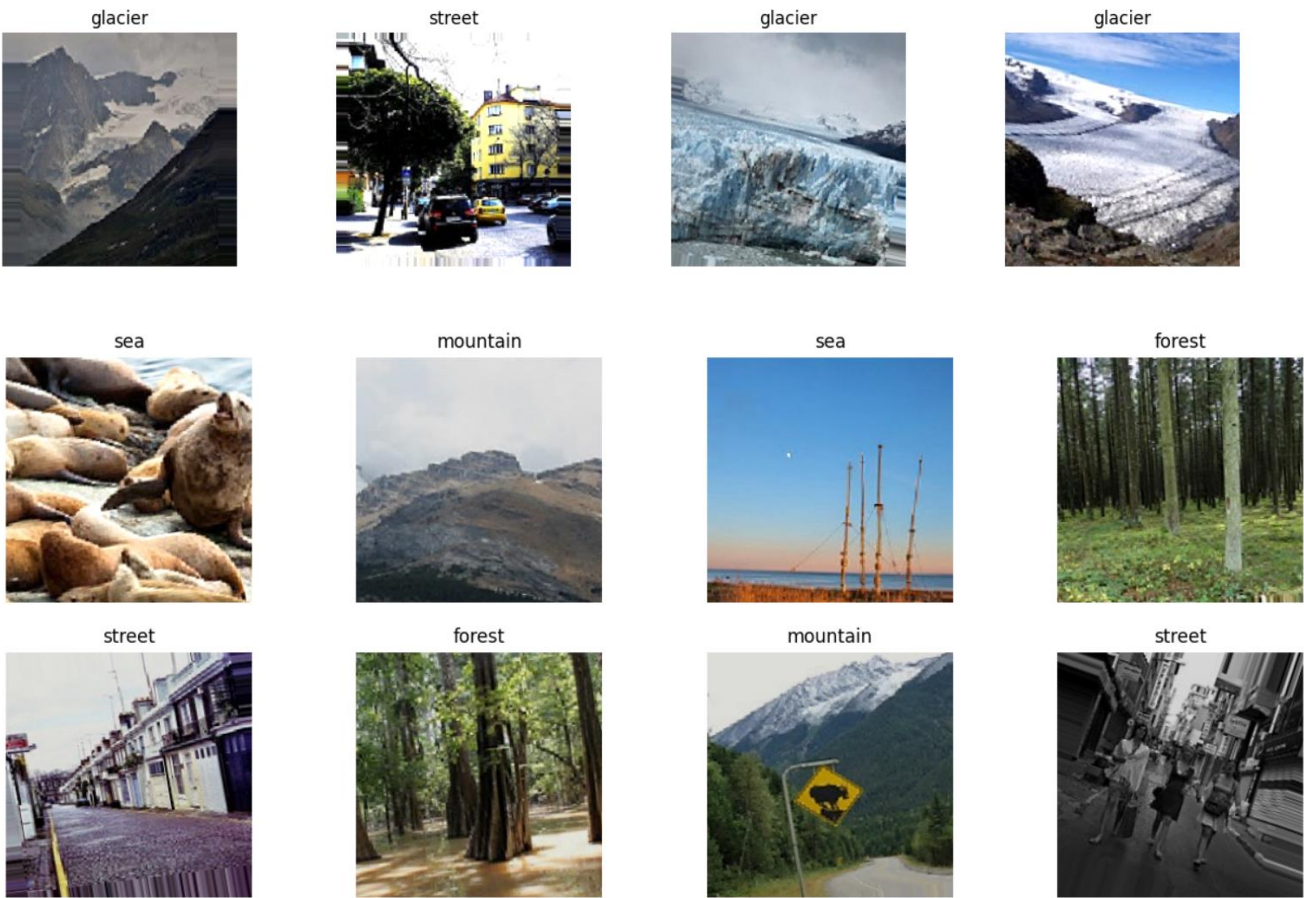


Figure 11: Original vs. augmented training samples.

6.4 Model Architecture & Training Configuration

MobileNetV2 (ImageNet weights) was used as a frozen feature-extraction backbone. A custom classification head — Global Average Pooling → Dense(512, ReLU) → Dropout(0.5) → Dense(6, Softmax) — was trained on top of it.

Parameter	Value
Backbone	MobileNetV2 (frozen, ImageNet weights)
Input Size	224 × 224 × 3
Optimizer	Adam (learning rate = 0.0001)
Loss Function	Categorical Crossentropy
Epochs	15 (EarlyStopping, patience = 5)
Batch Size	32
Callbacks	ModelCheckpoint (best val_accuracy), ReduceLROnPlateau

6.5 Results

Validation accuracy improved from 88.56% after the first epoch to a best checkpoint of 89.95%+ during training, and the model reached the following performance on the fully held-out test set:

Metric	Value
Test Accuracy	91.67%
Test Loss	0.2231
Weighted F1-score	0.92

Per-Class Precision, Recall and F1-score

Class	Precision	Recall	F1-score	Support
Buildings	0.95	0.90	0.93	437
Forest	0.99	0.99	0.99	474
Glacier	0.85	0.86	0.86	553
Mountain	0.89	0.83	0.86	525
Sea	0.93	0.98	0.95	510
Street	0.91	0.96	0.93	501

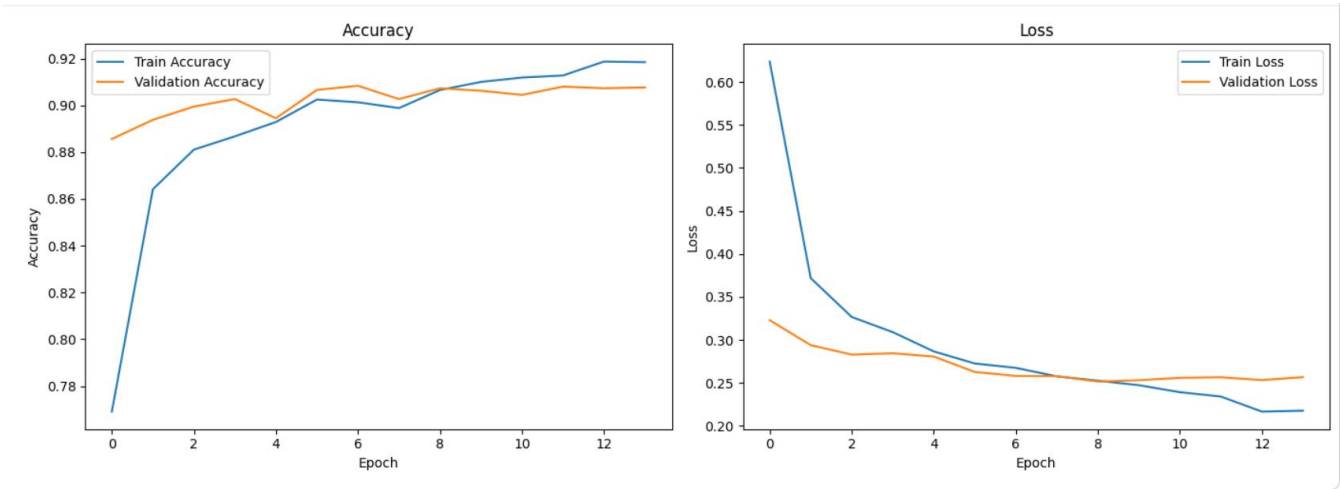


Figure 12: Training/validation accuracy and loss curves.

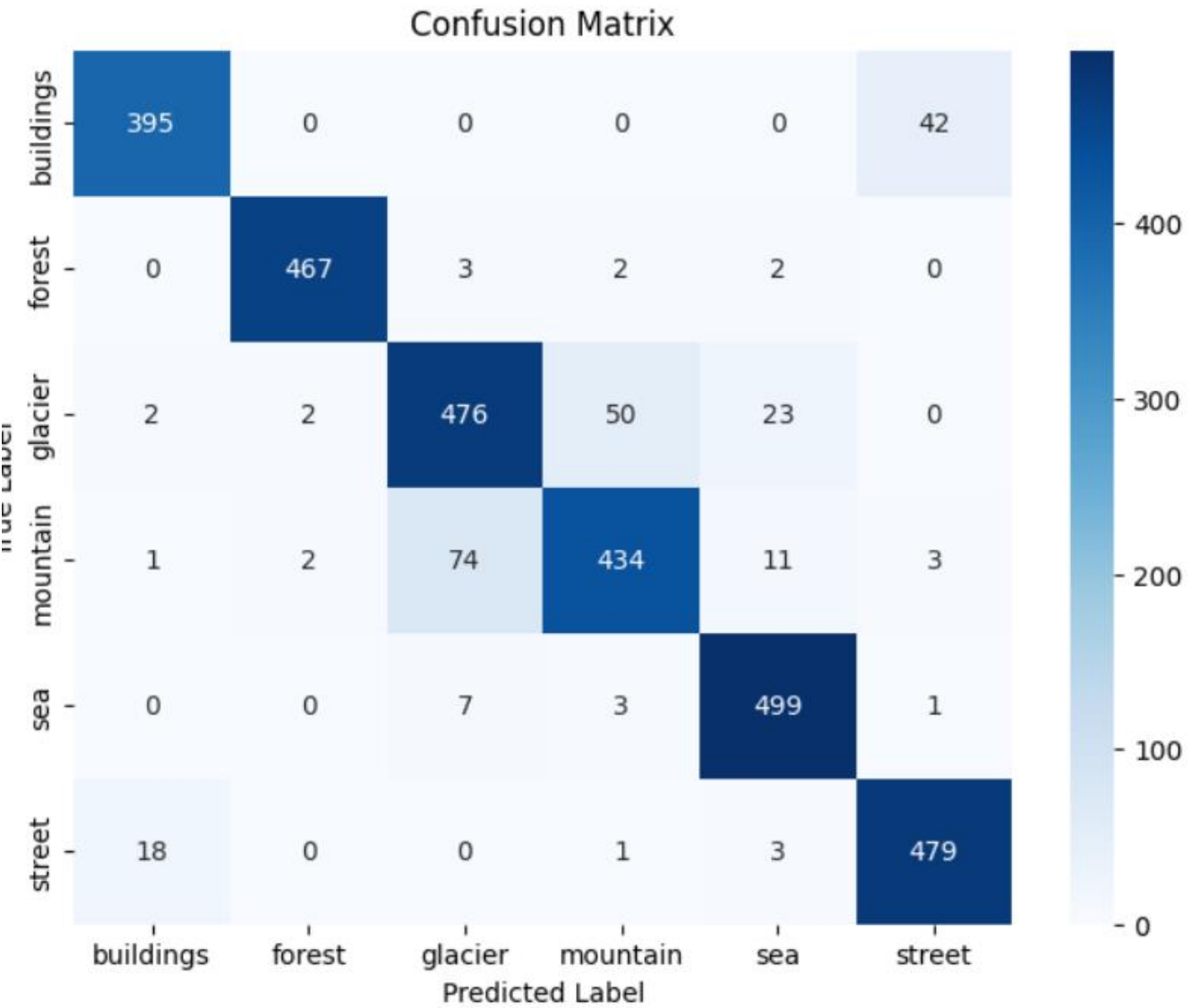


Figure 13: Confusion matrix for the six scene classes.

6.6 Grad-CAM Interpretability

Grad-CAM was used to visualise which image regions drove each prediction, using the last convolutional layer of the backbone (out_relu).

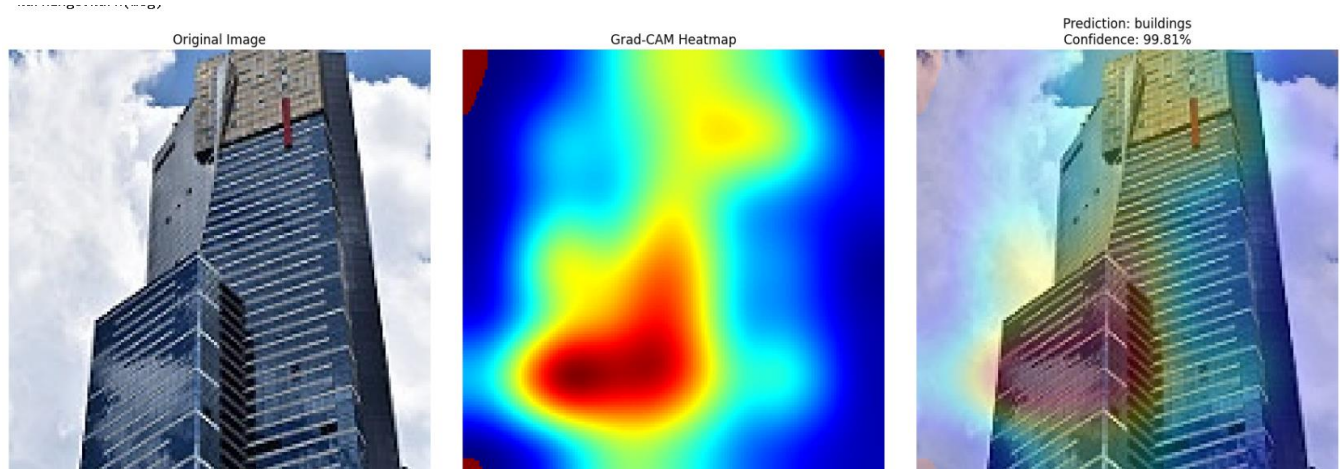


Figure 14: Correct prediction 1: buildings (99.81% confidence).

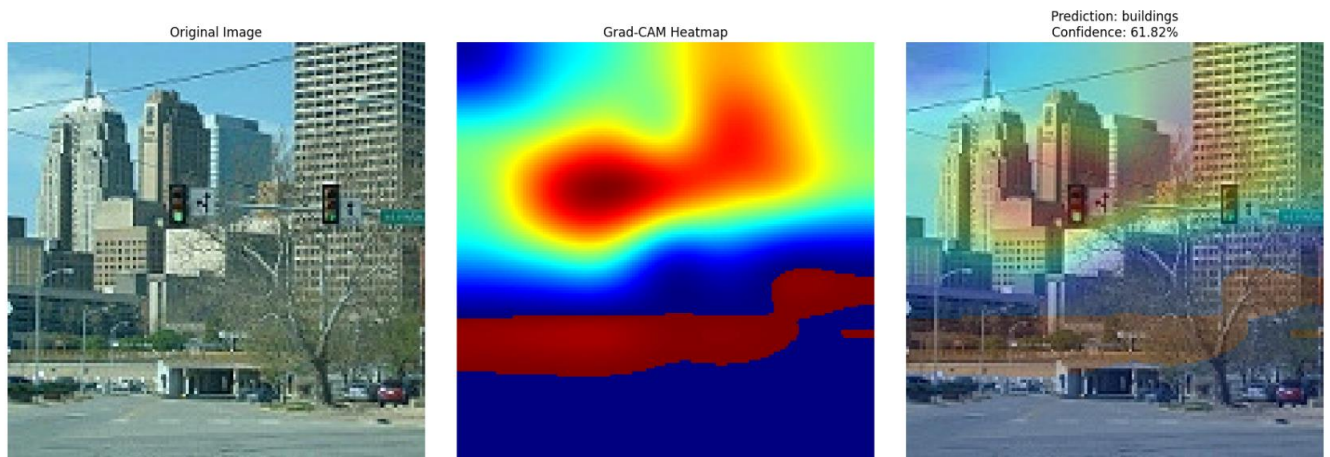


Figure 15: Correct prediction 2: buildings (61.82% confidence).

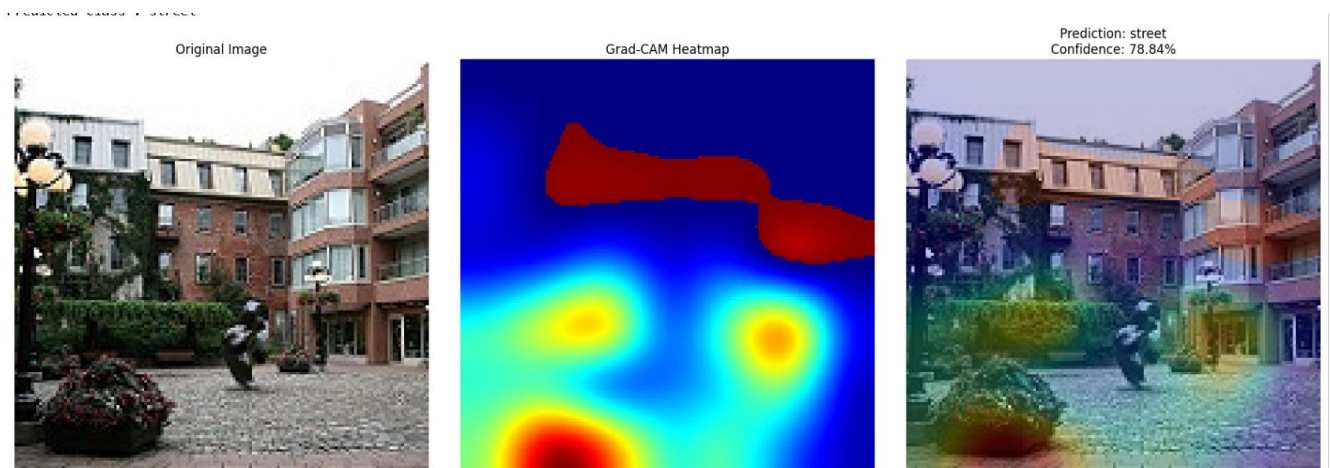


Figure 16: Incorrect prediction: true = buildings, predicted = street (78.84% confidence).

6.7 Discussion

- MobileNetV2 transfer learning reached 91.67% test accuracy while training only a lightweight classification head, confirming its suitability for resource-constrained robotics deployment.
- The confusion matrix and per-class report show the weakest recall for Mountain (0.83) and lowest precision for Glacier (0.85), consistent with the visual similarity between snowy mountain and glacier textures.
- The single analysed misclassification (buildings → street) had the model's Grad-CAM attention concentrated on the road/pavement area rather than the building structure, showing the model can latch onto the dominant visual context when a scene contains multiple plausible categories.
- Grad-CAM turns an otherwise black-box CNN decision into an inspectable, spatially-grounded explanation, which is valuable for debugging and building trust in a safety-relevant robotics pipeline.

6.8 Bonus: Interactive Deployment (Gradio UI)

Beyond the CCP's minimum requirements, the trained MobileNetV2 model was additionally wrapped in an interactive web application built with Gradio (gr.Blocks). The application loads the saved `Scene_Classification_MobileNetV2.keras` model, accepts an uploaded image, and returns the predicted scene class, the model's confidence score, and the full six-class probability breakdown through a custom-styled dashboard (dark theme, gradient title, card-based layout).

This deployment layer demonstrates that the trained classifier can be exposed as a usable end-user tool rather than remaining confined to a notebook, and is a realistic analogue of how a scene-classification module would be exposed to other components of an autonomous robotics stack (e.g., as a callable inference service).

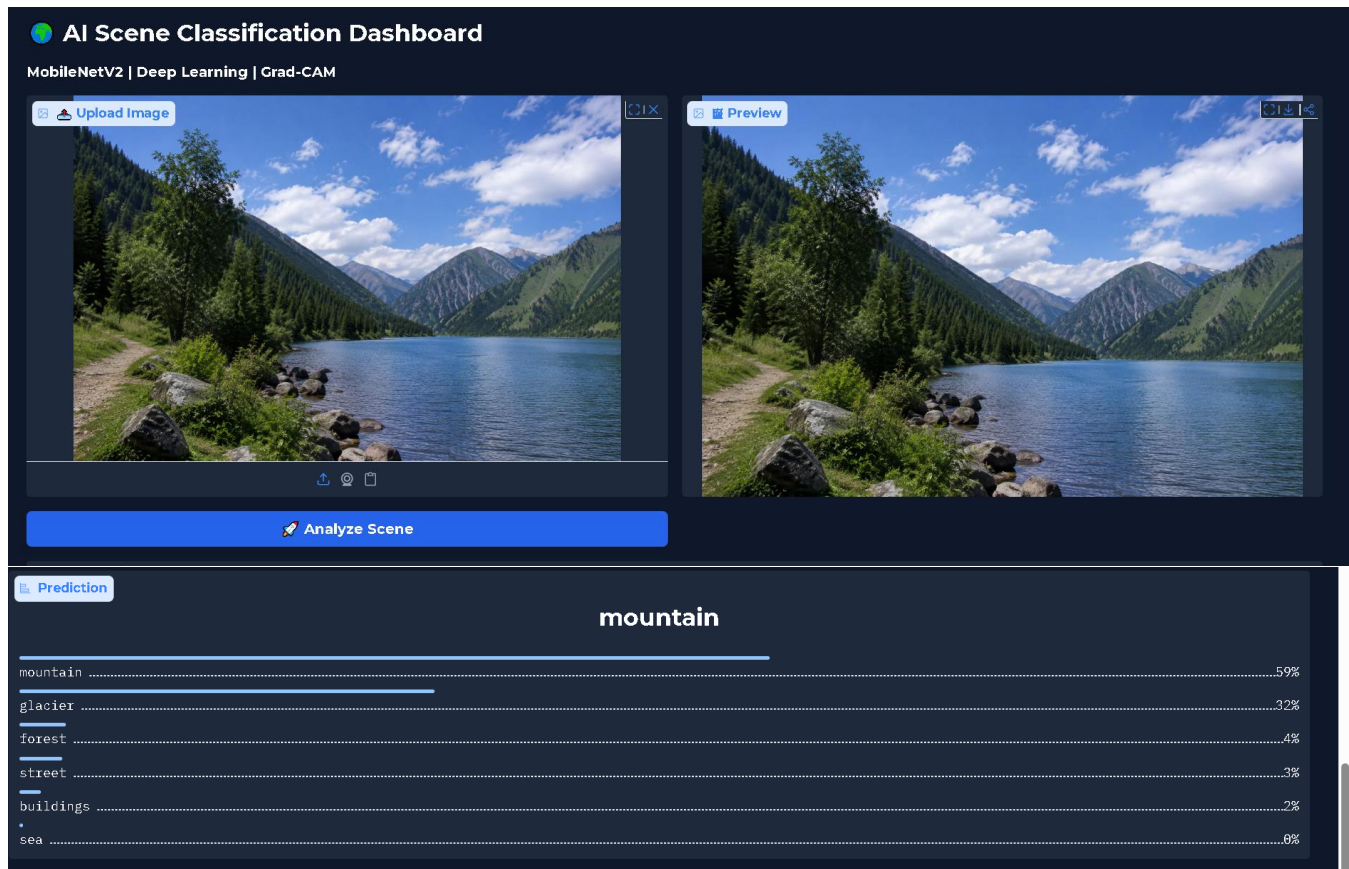


Figure 17: Interactive Gradio deployment of the trained scene-classification model.

5 — Critical Evaluation of the Pipeline (CLO-5)

7.1 Quantitative Summary Across All Tasks

Task	Key Metric	Value
T1 — Edge/Hough	Best Canny thresholds	100 / 200
T1 — Edge/Hough	Hough lines: full frame → ROI-masked	253 → 54
T2 — SIFT Matching	Keypoints (Img1 / Img2)	81,956 / 18,171
T2 — SIFT Matching	Good matches / Match quality ratio	4,130 / 5.04%
T2 — SIFT Matching	RANSAC inlier ratio	67.78%
T3 — Panorama	Final-stage matches (inliers/outliers)	318 (265 / 53)
T3 — Panorama	RANSAC inlier ratio	83.33%
T3 — Panorama	Seam position	x = 2172 px
T4 — CNN Classification	Test Accuracy / Test Loss	91.67% / 0.2231
T4 — CNN Classification	Weighted F1-score	0.92

7.2 Failure Analysis (≥ 2 modes per task)

Task 1 — Edge Detection & Hough Transform

- Unmasked Hough transform over-detects: running on the full Canny(100,200) edge map produced 253 lines, a large share of which come from non-road structures such as building edges and foliage rather than the road itself.
- Aggressive thresholding under-detects: the (150,250) Canny configuration suppressed faint or worn lane markings, particularly under shadow, leading to missed edges that a lower-threshold pass captured.

Task 2 — SIFT Feature Matching

- Unbalanced keypoint density: the two matched images produced very different keypoint counts (81,956 vs. 18,171), lowering the overall match quality ratio to 5.04% and indicating sensitivity to differences in texture/resolution between viewpoints.
- Ratio test is an imperfect filter: even after Lowe's ratio test, 1,331 of 4,130 "good" matches (32.2%) were still rejected by RANSAC as geometric outliers, showing that photometric similarity does not guarantee geometric consistency, especially in repetitive textures.

Task 3 — Panorama Construction

- Perspective distortion leaves empty canvas regions after warping, which had to be removed with manually hand-tuned crop coordinates rather than an automatic, content-aware crop — a solution that does not generalise to new image sequences without re-tuning.
- Exposure mismatch at the seam (measured $\Delta R \approx -39$, $\Delta G \approx -19$ between the two sides) required an explicit brightness-correction and blending step; without it, the panorama shows a visible vertical seam line.

Task 4 — CNN Scene Classification

- Visually overlapping classes are confused most often: Glacier/Mountain and Buildings/Street pairs share textures (snow-covered rock vs. ice; roads adjacent to buildings), driving down Glacier's precision (0.85) and Mountain's recall (0.83).
- Scenes with multiple plausible categories mislead the model: in the one closely examined error, Grad-CAM shows the network attending to the road/pavement rather than the building it was labelled as, because the road occupied a large portion of the frame.

7.3 Classical vs. Deep-Learning Trade-offs (Tasks 1–3 vs. Task 4)

Aspect	Classical Pipeline (T1–T3)	Deep Learning (T4)
Inference speed	Very fast, CPU-friendly per-frame operations (edge/Hough/SIFT)	Fast on GPU; noticeably slower per-batch on CPU
Training/tuning cost	No training; but heavy manual parameter tuning per image (thresholds, ROI, crop, seam)	One-off training run (~15 epochs); parameters learned automatically from data
Accuracy in this project	High under matched, controlled conditions (e.g., 83.33% RANSAC inlier ratio)	91.67% held-out test accuracy across 6 classes
Interpretability	Fully transparent — every step is a deterministic geometric transform	Requires post-hoc tools (Grad-CAM) to explain decisions
Data requirement	Works with as few as 2–3 raw images, no labels needed	Needed 11,266 labelled training images across 6 classes plus a pretrained backbone
Generalisation	Poor — thresholds/crops tuned for one scene often fail on a new lighting condition or layout	Better generalisation via learned features and augmentation, though still confused by visually similar classes

7.4 Limitations & Future Work

- No multi-band/Laplacian-pyramid blending was implemented for the panorama; only local brightness matching was used, which does not fully eliminate visible seams under strong exposure differences.
- The stitching pipeline was built and crop-tuned for a fixed 3-image left/center/right sequence; it does not yet generalise automatically to an arbitrary number of overlapping frames or aspect ratios.
- Only the CNN's classification head was fine-tuned while the MobileNetV2 backbone remained frozen; unfreezing and fine-tuning the deeper layers could further improve accuracy, particularly on the confused Glacier/Mountain and Buildings/Street pairs.
- Canny and Hough parameters were tuned manually per image; an adaptive/automatic thresholding scheme (e.g., median-based Canny sigma, or a learned line detector) would generalise better across varying lighting and road conditions.
- The four pipeline stages currently run independently; there is no single end-to-end metric evaluating, for example, whether classification accuracy is affected when run on a stitched panorama rather than a single frame.
- With more time and data, the next steps would be to: integrate all four stages into a single real-time pipeline (e.g., a ROS node), fine-tune deeper CNN layers, benchmark the manual stitching pipeline against OpenCV's cv2.Stitcher as a classical baseline, and evaluate the system on video sequences rather than static frames.

References

Dataset:

[1] P. Bansal, "Intel Image Classification," *Kaggle*, 2019. [Online]. Available: <https://www.kaggle.com/datasets/puneet6060/intel-image-classification> (Originally released for the Intel Image Classification Challenge on Analytics Vidhya.)

Classical Computer Vision Techniques:

[2] J. Canny, "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986.

[3] R. O. Duda and P. E. Hart, "Use of the Hough Transformation to Detect Lines and Curves in Pictures," *Communications of the ACM*, vol. 15, no. 1, pp. 11–15, 1972.

[4] D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[5] M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

Deep Learning:

[6] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[7] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626.

Software / Tools:

[8] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[9] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. [Online]. Available: <https://www.tensorflow.org>

[10] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, "Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild," *arXiv preprint arXiv:1906.02569*, 2019.

AI Tool Declaration (for Section 8):

"ChatGPT (OpenAI) was used to assist with debugging, code structuring, and drafting/formatting portions of this written report from the student's own experimental results and notebook outputs."